文本分类实验报告

自然语言处理作业一

姓名：姚智博　　学号：2313557

这次作业要完成的任务是：给定一篇新闻，预测它属于商业、政治还是体育。实验比较两种词袋表示、三种平均词向量表示，以及 BERT 微调。前五组使用 Logistic Regression，最后一组直接微调 BERT 的全部参数。所有结果均在同一批 1,152 篇 NYT 测试新闻上计算。

任务	文本表示	分类器
Task 1	Binary BoW、Word Frequency	Logistic Regression
Task 2	GloVe 100d、AG Word2Vec 100d、NYT Word2Vec 100d	Logistic Regression
Task 3	BERT-base-uncased，max_length=64	BERT 三类别分类头

实验流程

阶段	本次具体处理
准备数据	读取 CSV，检查空文本、标签和重复记录，固定一次 80%/10%/10% 划分。
建立表示	训练词表或词向量，或者使用指定的公开词向量与 BERT tokenizer。
训练与选参	只在训练集拟合参数，用验证集选 Logistic Regression 配置和 BERT checkpoint。
最终评价	预测测试集，保存逐篇标签，再计算 Accuracy、Macro-F1 与混淆矩阵。

主要结果

Binary BoW 的测试 Accuracy 为 98.96%，Macro-F1 为 97.79%。Word Frequency 的 Accuracy 相同，Macro-F1 为 97.52%。三组平均词向量中，NYT Word2Vec 最好；BERT 的测试 Macro-F1 为 96.50%。后文把这些结果与词项覆盖率、类别表现、输入截断和具体错误对应起来分析。

小tip：图中 External 表示 AG 语料，Target 表示 NYT 训练语料。代码节选的源文件位置标在各段代码下方。

---

1. 数据检查与类别分布

HW-1/nyt.csv 有 text 和 label 两列，共 11,519 篇新闻。HW-1/ag.csv 有 90,000 篇新闻，只有 text 列，用于训练外部语料的 Word2Vec。两份文件均没有空文本；AG 与 NYT 没有完全相同的文本。

类别	全部样本	训练集	验证集	测试集
business	1429	1137	155	137
politics	1451	1127	172	152
sports	8639	6951	825	863

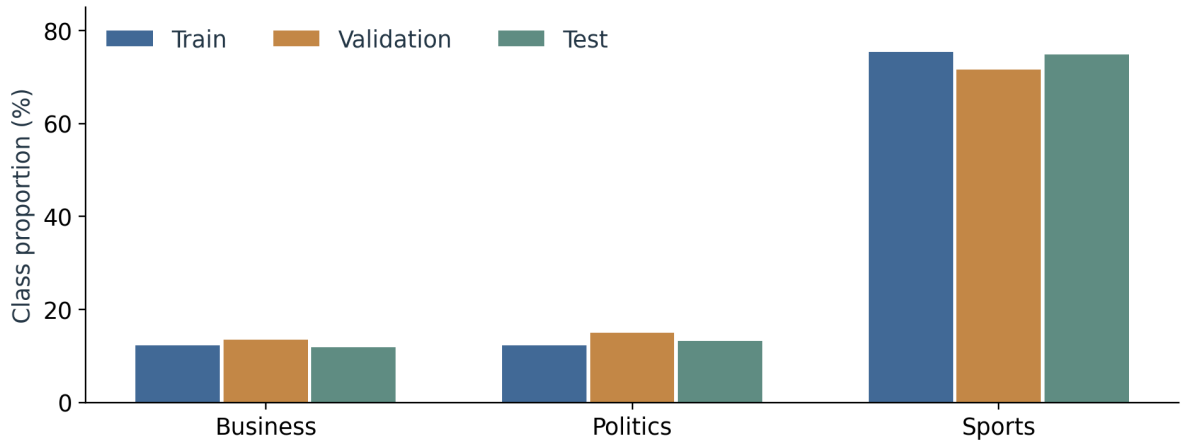


图 1 各划分中三个类别的比例

体育新闻占全部样本的 75.00%，明显多于另外两类。直接打乱后按比例切分，三个子集的类别比例仍然接近。评价时同时观察 Macro-F1，可以更清楚地看到商业和政治两类的表现。

检查项	实际结果
NYT 重复行	72 条；相同文本没有不同标签。
AG 重复行	29 条。
训练与测试重复	共享 15 个不同的完全相同文本，后文另做过滤后的评价。
普通分词后的长度	均值 656.1，中位数 690，90% 分位数 970。

数据检查结果保存在 results/data_audit.json。原始文件的 SHA-256 一并保存，后续可以确认数据文件没有被改动。

2. 固定划分与分词代码

随机种子设为 42。先生成包含全部原始行号的随机排列，再在 80% 和 90% 的位置切开。训练、验证和测试集分别有 9,215、1,152、1,152 篇。测试行号从 0 开始，不包括 CSV 的表头。

```
permutation = np.random.default_rng(SEED).permutation(len(frame))
n_train, n_train_val = int(.8*len(frame)), int(.9*len(frame))
indices = dict(train=permutation[:n_train],
validation=permutation[n_train:n_train_val],
               test=permutation[n_train_val:])
split_path = ROOT/'results'/'split_indices.json'
serial = {key: values.tolist() for key, values in indices.items()}
if split_path.exists():
    if json.loads(split_path.read_text(encoding='utf-8')) != serial:
        raise ValueError('Existing split differs; do not silently change the
split')
else: write_json(split_path, serial)
```

来源：`src/common.py 第 33 至 42 行`

permutation 只改变样本顺序，原始 CSV 不需要重写。保存 indices 后，词袋、词向量和 BERT 都通过同样的行号读取样本。程序会核对已有的划分文件，保证后续运行沿用相同的测试样本。

普通分词器

```
TOKEN = re.compile(r"[a-z]+(?:'[a-z]+)?|[0-9]+(?:\.[0-9]+)?")

def tokenize(text):
    """Lowercase English words, contractions and numbers; no external NLTK
download."""
    return TOKEN.findall(str(text).lower())
```

来源：`src/common.py 第 13 至 18 行`

输入或设置	作用
Apple apple, DON'T 12.5!	得到 apple、apple、don't、12.5 四个词项。重复词不会在这里删除。
转为小写	Apple 和 apple 共用同一个词项。
保留撇号与数字	缩写和数字可以进入训练词表。
不删停用词 不提取词干	保持基础词袋和平均词向量方法的处理一致。

本实验使用正则表达式进行英文分词，统一处理大小写、缩写和数字。遇到连字符时通常会分成多个词，不进行词形还原。BERT 的输入则由对应的 WordPiece tokenizer 处理。

---

3. Task 1 词袋向量的构建

两种词袋共用一个训练词表，大小为 66,423。验证或测试中从未出现在训练集的词不会新增维度。例如，训练词表只有 apple 和 banana 时，下面的文档都被表示为两个维度。

文档	Binary 向量	Frequency 向量
apple apple banana	[1, 1]	[2, 1]
banana	[0, 1]	[0, 1]
apple unseen	[1, 0]	[1, 0]；unseen 未进入训练词表

Binary 的第 i 维为 $1[c_i > 0]$，只看出现与否；Frequency 的第 i 维为 c_i，保留原始次数。这两种表示均不做长度归一化。

```
vectorizer = CountVectorizer(tokenizer=tokenize, token_pattern=None,
lowercase=False,
                             min_df=1, dtype=np.float64)
vectorizer.fit(frame.text.iloc[split['train']])
counts = vectorizer.transform(frame.text)
common_preprocess_seconds = time.perf_counter()-started
for name in ['bow_binary', 'bow_frequency']:
    start = time.perf_counter()
    if name == 'bow_binary':
        features = counts.copy()
        features.data[:] = 1.
```

```
    else:
        features = counts
    finish(name, features, frame, y, split,
           {'vocabulary_size': len(vectorizer.vocabulary_), 'min_df': 1,
'ngrams': [1,1],
            'preprocess_seconds': common_preprocess_seconds+time.perf_counter()-
start, 'representation': name,
            'vocabulary_fit': 'NYT train only', 'feature_dimension':
features.shape[1]}, start-common_preprocess_seconds,
```

来源：`src/train_classical.py 第 59 至 74 行`

代码中的几个细节

CountVectorizer 在训练文本上建立词表，再用固定词表转换验证和测试文本。[4] token_pattern=None 表示使用自定义分词器；lowercase=False 是因为 tokenize 已经转为小写。

counts 为稀疏矩阵，只有非零词项需要存储。Binary 先复制 counts，再把非零值全部置为 1；这样可以同时保留原始计数矩阵，供 Frequency 实验使用。min_df=1 表示保留所有训练词项，没有人为截断到固定的 top-k 词表。

---

4. Logistic Regression 训练与模型选择

对文档向量 x，分类器为每个类别计算线性分数 $z_k = w_k^T x + b_k$，通过 softmax 得到三个类别的概率，取概率最高的类别作为预测。训练使用 L2 正则；C 越小，正则约束越强。

设置	取值或规则
分类器	LogisticRegression，lbfgs，max_iter=3000，tol=1e-4。
正则参数 C	0.01、0.1、1、10。
类别权重	None 或 balanced，共八种候选组合。
选参指标	验证集 Macro-F1 优先，其次 Accuracy；完全相同时保留先出现的组合。
最终拟合范围	使用训练集拟合分类器，验证集用于选择配置。

```
for weight in [None, 'balanced']:
    for c in CS:
        model = LogisticRegression(C=c, class_weight=weight, solver='lbfgs',
                                   max_iter=3000, tol=1e-4, random_state=SEED)
        with warnings.catch_warnings(record=True) as caught:
            warnings.simplefilter('always', ConvergenceWarning)
            model.fit(features[split['train']], y[split['train']])
        if any(issubclass(w.category, ConvergenceWarning) for w in caught):
            raise RuntimeError(f'LR did not converge: C={c}, class_weight=
{weight}')
        val = metrics(y[split['validation']],
model.predict(features[split['validation']]))
        trial = {'C': c, 'class_weight': weight, 'validation_accuracy':
val['accuracy'],
                 'validation_macro_f1': val['macro_f1'], 'n_iter':
model.n_iter_.tolist()}
```

```
        trials.append(trial)
        print(trial, flush=True)
        rank = (val['macro_f1'], val['accuracy'])
```

来源：`src/train_classical.py 第 23 至 37 行`

balanced 的类别权重为 $N_{train} / (3 \times N_k)$。由于训练集体育样本多，商业和政治类样本会获得更大的权重。是否使用 balanced 由验证结果决定。

通过捕获 ConvergenceWarning 检查模型收敛情况。本次所有候选配置均正常收敛，迭代次数保存在各自的结果 JSON 中。

---

5. 词袋模型的验证结果与选参依据

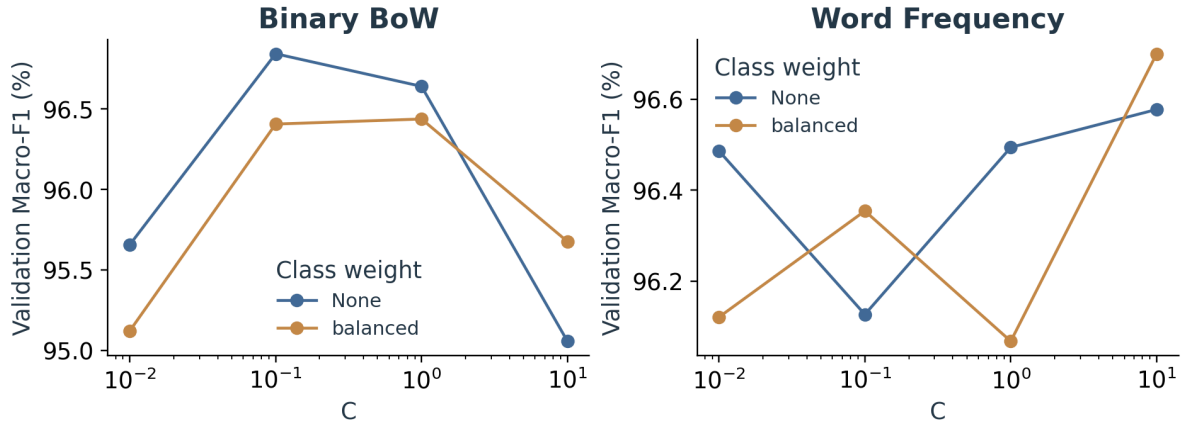


图 2 两种词袋模型的验证 Macro-F1 随 C 的变化

C	类别权重	Binary 验证 F1	Frequency 验证 F1
0.01	None	95.66%	96.49%
0.1	None	96.84%	96.13%
1.0	None	96.64%	96.49%
10.0	None	95.06%	96.58%
0.01	balanced	95.12%	96.12%
0.1	balanced	96.41%	96.35%
1.0	balanced	96.44%	96.07%
10.0	balanced	95.68%	96.70%

Binary 选中的配置是 C=0.1、class_weight=None；Frequency 选中 C=10.0、class_weight=balanced。两组配置分别对应上表中最高的验证集 Macro-F1。

两种向量的尺度不同：Binary 的非零值都为 1，Frequency 中较长文章或高频词会产生更大的值。从曲线看，两种表示的最佳 C 不同，增大 C 后分数也有下降的情况。

---

6. Task 2 词向量来源与 GloVe 加载

词袋为每个词分配一个独立维度，词向量则用固定长度的数值表示一个词。GloVe 根据大规模语料中的全局词共现统计学习向量；Word2Vec 则通过局部上下文训练。本题统一用 100 维词向量，再对每篇新闻中的有效词向量取平均。[1][2]

实验	词向量来源	是否自行训练
GloVe	glove.6B.100d.txt；400,000 个词项	否，直接读取公开向量。
AG Word2Vec	HW-1/ag.csv 的全部文本	是，只用 AG 文本。
NYT Word2Vec	固定 NYT 训练集的 9,215 篇新闻	是，使用训练集文本。

读取 GloVe 文件

```
vectors = {}
with path.open(encoding='utf-8') as source:
    for line in source:
        word, _, values = line.partition(' ')
        vector = np.fromstring(values, sep=' ', dtype=np.float32)
        if vector.shape != (100,): raise ValueError('Invalid GloVe dimension')
        vectors[word] = vector
metadata.update({'source': 'Stanford glove.6B.100d.txt',
'loaded_vocabulary_size': len(vectors),
                 'vocabulary_filter': 'none; fixed public pretrained vectors'})
```

来源：`src/train_classical.py 第 98 至 106 行`

GloVe 文本文件的一行由一个词和 100 个浮点数构成。partition 取出词项，np.fromstring 将后面的数字转换为向量。读取时检查向量是否包含 100 个数，便于发现文件格式错误。

GloVe 使用公开预训练向量，NYT Word2Vec 则仅使用训练集语料，两者均不使用测试标签。

未登录词的处理

某个词不在对应词向量词表中时，跳过这个词。如果一篇文档没有任何有效词，返回全零向量，保证分类器仍能接收固定长度的输入。本次三组词向量的测试集都没有出现全零文档。

---

7. Word2Vec 的训练代码与语料规模

两组 Word2Vec 都使用 Gensim 的 Skip-gram 实现，训练参数相同，语料分别取 AG 文本和 NYT 训练文本。[2][5]

```
if name == 'w2v_ag':
    corpus = [tokenize(text) for text in pd.read_csv(ROOT/'HW-1'/'ag.csv').text]
    corpus_name = 'AG News only'
else:
    corpus = [docs[ix] for ix in split['train']]
    corpus_name = 'NYT training split only'
settings = dict(vector_size=100, window=5, min_count=2, sg=1, hs=0, negative=5,
                sample=1e-3, epochs=10, workers=1, seed=SEED,
hashfxn=stable_hash,
```

```
                sorted_vocab=1)
model = Word2Vec(sentences=corpus, **settings)
vectors = model.wv
model.save(str(ROOT/'models'/f'{name}.model'))
metadata.update({'corpus': corpus_name, 'corpus_documents': len(corpus),
                 'gensim_version': version('gensim'),
```

来源：`src/train_classical.py 第 110 至 123 行`

参数	设置	含义
vector_size	100	词向量维度。
window	5	上下文窗口的最大范围。
min_count	2	忽略语料中只出现一次的词。
sg / negative	1 / 5	采用 Skip-gram 和负采样。
sample	0.001	对高频词进行下采样。
epochs / workers	10 / 1	训练十轮，单线程便于复现。
seed / hashfxn	42 / stable_hash	固定随机初始化和词项哈希。

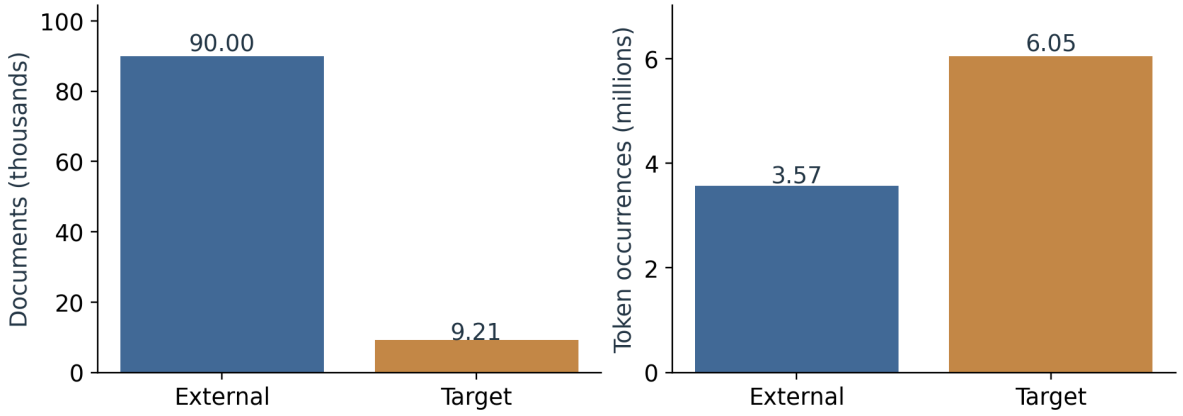


图 3 两份 Word2Vec 训练语料的文档数和词项出现次数

AG 的词表有 38,140 个词，NYT 训练语料词表有 46,322 个词。AG 的文档数更多，但文本较短，总词项数约 356 万；NYT 训练集总词项数约 605 万。NYT 单篇新闻更长，因此文档数较少，总词项数却更高。

---

8. 从词向量得到文档向量

```
def mean_embeddings(documents, vectors):
    features = np.zeros((len(documents), 100), dtype=np.float32)
    covered, total, zero = 0, 0, 0
    for i, tokens in enumerate(documents):
        valid = [vectors[word] for word in tokens if word in vectors]
        covered += len(valid)
        total += len(tokens)
        if valid: features[i] = np.mean(valid, axis=0)
        else: zero += 1
    return features, {'covered_tokens': covered, 'total_tokens': total,
                      'token_coverage': covered/max(total,1),
'zero_vector_documents': zero}
```

来源：`src/train_classical.py 第 77 至 87 行`

features 预先建立为“文档数 × 100”的零矩阵。对第 i 篇文档，valid 收集有向量的词项；同一个词出现两次，就加入两次向量。np.mean 沿 axis=0 求平均，得到一个 100 维行向量。

二维演示 仅解释公式	**数值**
假设词向量	apple=[1,0]；banana=[0,1]。
文档	apple apple banana unknown。
有效词项数	3；unknown 不参与平均。
平均向量	([1,0]+[1,0]+[0,1])/3 = [2/3,1/3]。

上表用二维向量演示平均过程，实际实验使用 100 维向量。有效词项共有 3 个，因此求平均时除以 3。

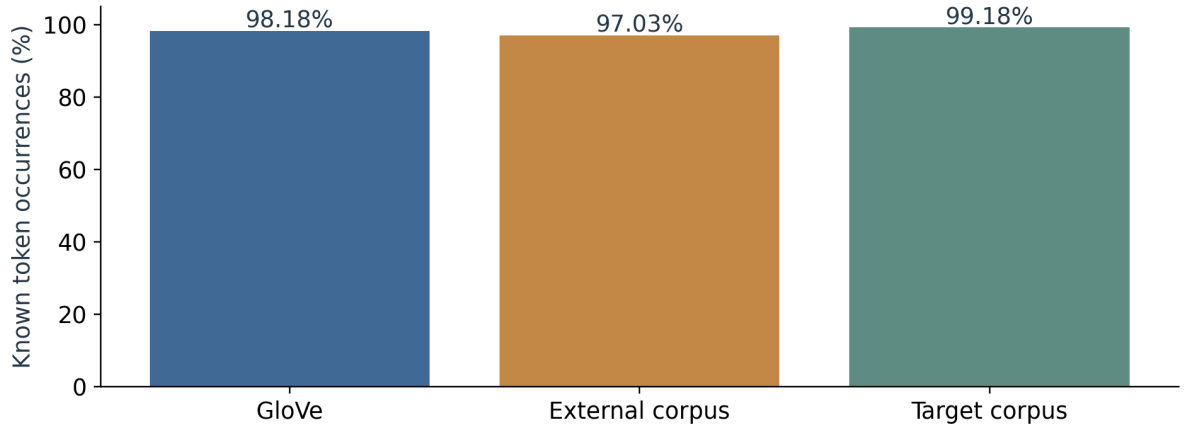


图 4 测试集有效词项覆盖率 按词项出现次数计算

覆盖率为“有向量的词项出现次数 / 全部词项出现次数”，重复词按出现次数计入。得到平均向量后，用 StandardScaler 调整各维度尺度；均值和标准差只在训练集拟合，再应用于验证和测试向量。三组文档向量使用相同的分类器选参规则。

9. 三组平均词向量的测试结果

方法	测试覆盖率	Accuracy	Macro-F1	C
GloVe 100d	98.18%	98.61%	96.68%	1.0
Word2Vec AG	97.03%	98.26%	95.97%	10.0
Word2Vec NYT	99.18%	98.78%	96.85%	0.1

三组模型最后都选择 class_weight=None。NYT Word2Vec 的 Macro-F1 比 AG Word2Vec 高 0.89 个百分点，测试覆盖率也更高。NYT 语料的主题与测试数据更接近，而且总词项数更多，可能是其分类效果更好的原因。

与词袋方法的对照

三组平均词向量的 Macro-F1 都低于两种词袋。平均操作把主题词、背景词和通用词混在同一个向量里，原本明显的主题线索可能被稀释。例如，词袋能单独保留 football 是否出现，平均词向量却需要把它和全文其他词一起平均。

表示	文档特征维度	主要保留的信息
Binary / Frequency	66,423，稀疏	各词是否出现或出现多少次。
平均词向量	100，稠密	有效词的分布式向量平均。
BERT	分类头接收 768 维隐藏表示	在指定输入长度内编码上下文。

平均词向量把文档压缩为 100 维，减少了特征数量。词向量先通过上下文学习词语表示，再由 Logistic Regression 根据新闻标签学习分类边界。

---

10. Task 3 BERT 输入与分类头

使用 google-bert/bert-base-uncased 预训练模型。[3][6] 文本先由对应的 WordPiece tokenizer 转为 token ID，再输入预训练编码器和三类别分类头。普通分词器的词项与 WordPiece 子词不是同一种单位。

```
tokenizer = BertTokenizerFast.from_pretrained(str(local), local_files_only=True)
encodings = tokenizer(frame.text.tolist(), truncation=True, padding='max_length',
                      max_length=64, return_tensors='pt')
full_lengths = np.array([len(ids) for ids in tokenizer(frame.text.tolist(),
truncation=False)['input_ids']])
```

来源：`src/train_bert.py 第 57 至 60 行`

输入项	作用
input_ids	词或子词及特殊标记的编号。
attention_mask	区分有效输入与补齐位置。
token_type_ids	区分句段；本次按单段新闻输入。

输入项	作用
labels	business=0、politics=1、sports=2。

```
class EncodedDataset(Dataset):
    def __init__(self, encodings, labels, indices):
        self.encodings, self.labels, self.indices = encodings, labels, indices
    def __len__(self): return len(self.indices)
    def __getitem__(self, i):
        index = self.indices[i]
        return {**{key: value[index] for key, value in self.encodings.items()},
                'labels': torch.tensor(self.labels[index], dtype=torch.long)}
```

来源：`src/train_bert.py 第 16 至 23 行`

EncodedDataset 按保存的原始行号取出编码和标签，因此 BERT 与传统模型的划分完全一致。labels 使用 torch.long，对应三分类的交叉熵损失。训练 DataLoader 打乱批次顺序，验证和测试按固定顺序读取。

max_length=64 包含 [CLS] 和 [SEP]，正文最多约 62 个子词；短文本补齐，长文本截断。测试集中 99.91% 的完整序列超过 64，完整序列平均长度为 834.7 个 WordPiece。这说明几乎所有测试新闻都只输入了开头部分，和读取全文的词袋、平均词向量存在信息量差异。

---

11. BERT 全参数微调与优化步骤

```
model = BertForSequenceClassification.from_pretrained(str(local), num_labels=3,
            id2label=dict(enumerate(LABELS)), label2id={label:i for i,label in
enumerate(LABELS)},
            local_files_only=True, attn_implementation='eager').to(device)
assert all(parameter.requires_grad for parameter in model.parameters()), 'Must
fine-tune full BERT'
# Standard AdamW: exempt biases and LayerNorm parameters from weight decay.
```

来源：`src/train_bert.py 第 67 至 71 行`

模型有 109,484,547 个可训练参数。分类头重新初始化，预训练主体也参与更新。训练前通过 requires_grad 检查全部参数的更新状态。

项目	实际配置
训练轮数与 batch	3 epochs；batch size=32；每轮 288 个 batch，最后一批 31 条。
优化器	AdamW，学习率 2e-5，weight decay=0.01。
学习率计划	计划 864 个训练 batch，前 87 步 warmup，之后线性衰减。
不做权重衰减的参数	bias 与 LayerNorm.weight。
梯度处理	FP16 AMP，GradScaler，梯度范数裁剪为 1.0。
运行设备	NVIDIA GeForce RTX 4060 Laptop GPU。

```
optimizer.zero_grad(set_to_none=True)
with autocast(): loss = model(**batch).loss
scaler.scale(loss).backward()
scaler.unscale_(optimizer)
torch.nn.utils.clip_grad_norm_(model.parameters(),1.)
old_scale = scaler.get_scale()
scaler.step(optimizer); scaler.update()
if scaler.get_scale() >= old_scale: scheduler.step()
total_loss += loss.item()*len(batch['labels']); n_seen += len(batch['labels'])
```

来源：`src/train_bert.py 第 88 至 96 行`

每个 batch 先计算损失并反向传播。AMP 下先把梯度缩放值还原，再裁剪范数，随后更新参数。如果 GradScaler 因溢出跳过了参数更新，学习率调度也不向前走。训练模式打开 dropout，验证时切到 eval 模式。

---

12. BERT 三轮训练记录与模型选择

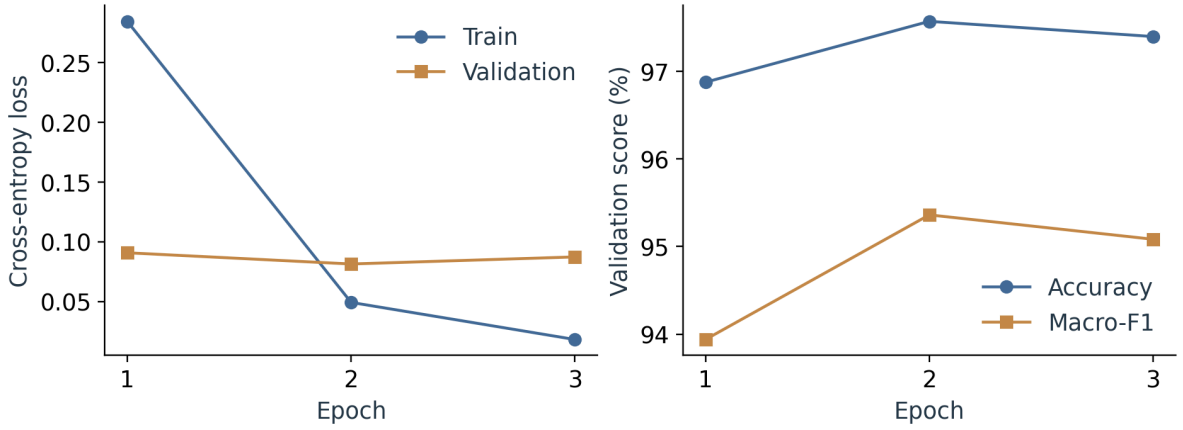


图 5 三轮训练损失与验证集指标

Epoch	训练损失	验证损失	验证 Accuracy	验证 Macro-F1
1	0.2847	0.0908	96.88%	93.94%
2	0.0492	0.0814	97.57%	95.36%
3	0.0181	0.0873	97.40%	95.08%

BERT training log excerpt

Source: logs/train_bert.log | Actual file excerpt

```
Device cuda, batch 32, steps per epoch 288
epoch 1/3 step 288/288 loss 0.28474 elapsed 50.8s
epoch 2/3 step 288/288 loss 0.04921 elapsed 53.0s
epoch 3/3 step 288/288 loss 0.01809 elapsed 50.7s
FINISHED BERT accuracy=0.983507 macro-F1=0.965023
```

图 6 训练日志的关键输出，已对原始日志进行排版整理。来源：logs/train_bert.log

第二轮验证 Macro-F1 为 95.36%，第三轮降到 95.08%；训练损失却继续下降，出现了过拟合的迹象。训练完成三轮后，根据验证集 Macro-F1 选择第 2 轮 checkpoint 进行测试。

日志节选包含每轮完成时的进度行和最终测试分数。完整日志中的模型加载、验证统计与 checkpoint 保存信息可在源文件中查看。

---

13. 在测试集上计算 Accuracy 和 Macro-F1

测试评价发生在模型选择完成之后。传统模型调用 predict(features[split["test"]])；BERT 先加载验证集最优模型，再用 test DataLoader 预测。模型进入 eval 模式，并在 torch.no_grad 下执行，关闭 dropout 和梯度计算。

```
def metrics(y, pred):
    return {'accuracy': float(accuracy_score(y, pred)),
            'macro_f1': float(f1_score(y, pred, labels=range(3), average='macro',
zero_division=0)),
            'classification_report': classification_report(y, pred,
labels=range(3),
                  target_names=LABELS, output_dict=True, zero_division=0),
            'confusion_matrix': confusion_matrix(y, pred,
labels=range(3)).tolist()}
```

来源：`src/common.py 第 45 至 50 行`

Accuracy 是预测正确的篇数除以测试篇数。对类别 k，$F1_k = 2TP_k / (2TP_k + FP_k + FN_k)$。Macro-F1 是三个类别 F1 的算术平均。计算时指定三个类别；某类的分母为零时，该类 F1 记为零。

Test metric verification output

Source: logs/report_metric_check.log | Actual file excerpt

```
SAVED TEST PREDICTIONS - METRIC CHECK
Test samples: 1152

Model                    Accuracy   Macro-F1   Errors
Binary BoW               0.989583   0.977855       12
Word Frequency           0.989583   0.975208       12
GloVe 100d               0.986111   0.966792       16
Word2Vec External        0.982639   0.959650       20
Word2Vec Target          0.987847   0.968546       14
BERT                     0.983507   0.965023       19

PASS: all six metrics match the saved results.
PASS: all models use identical test row IDs.
```

图 7 测试指标核算的关键输出，已对原始日志进行排版整理。来源：logs/report_metric_check.log

核算时逐项检查测试行号和真实标签，再把预测标签映射为 0、1、2，重算全部指标与混淆矩阵。六组结果与各模型 JSON 完全一致。

---

14. 六组模型的最终测试结果

方法	Accuracy	Macro-F1	错误数	流程秒数
Binary BoW	98.96%	97.79%	12	17.7

方法	Accuracy	Macro-F1	错误数	流程秒数
Frequency BoW	98.96%	97.52%	12	46.2
GloVe 100d	98.61%	96.68%	16	42.6
Word2Vec AG	98.26%	95.97%	20	256.1
Word2Vec NYT	98.78%	96.85%	14	418.2
BERT	98.35%	96.50%	19	161.9

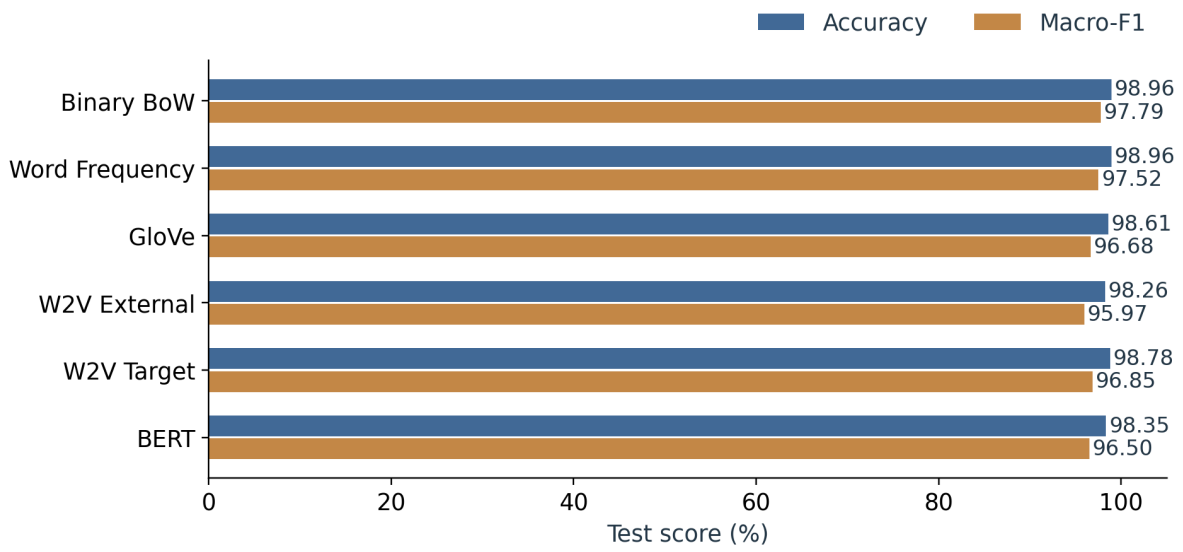


图 8 同一测试集上的 Accuracy 和 Macro-F1　横轴从零开始

Binary 和 Frequency 都错分 12 篇，所以 Accuracy 相同；Binary 的 Macro-F1 高 0.26 个百分点。三种平均词向量中，NYT Word2Vec 的两个指标都最高。BERT 错分 19 篇，比 Binary 多 7 篇，Binary 是本次测试中 Macro-F1 最高的模型。

时间包含表示构建、分类器训练和评价；Word2Vec 包含词向量训练，BERT 包含三轮训练、验证和模型保存。下载和环境安装不计入。由于传统模型和 BERT 使用不同计算设备，运行时间仅作为本次实验的参考。

15. 各类别表现与完整混淆矩阵

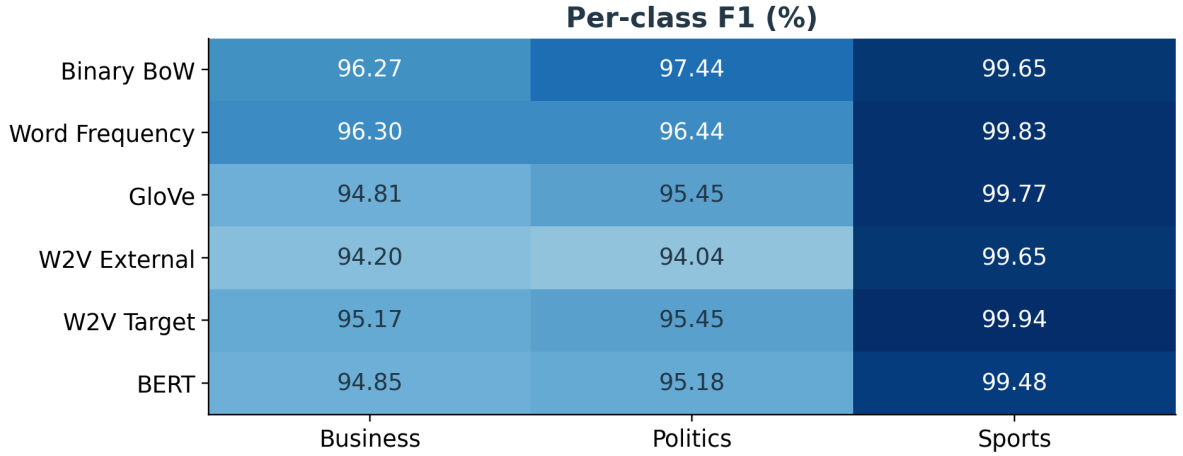


图 9 六组模型的逐类 F1　颜色范围为 90% 至 100%

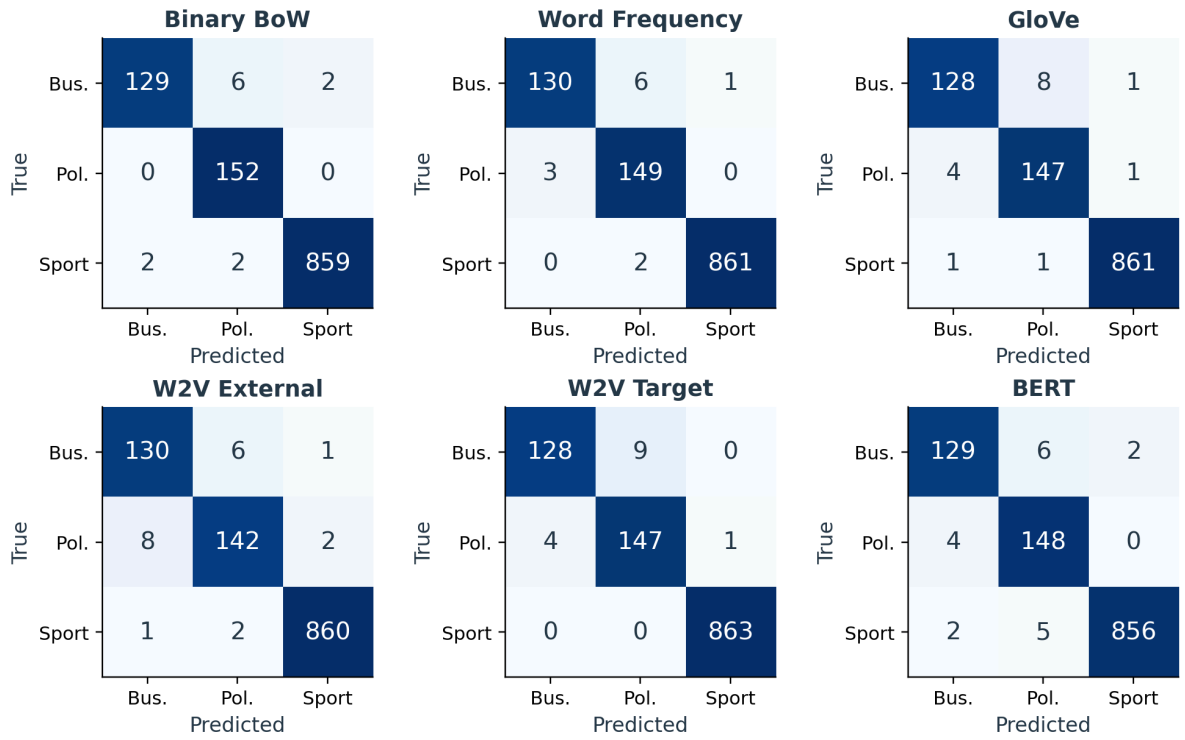


图 10 六组测试混淆矩阵 行是真实类别 列是预测类别

所有模型的体育类 F1 都在 99% 以上，商业和政治类相对更难。Binary 正确识别了全部 152 篇政治新闻，同时把 6 篇商业和 2 篇体育新闻预测为政治；政治类 recall 为 100%，precision 为 95.00%。

Binary 和 BERT 的商业类正确数都为 129。BERT 比 Binary 多出的 7 个错误，来自政治类多错 4 篇和体育类多错 3 篇。矩阵数字为样本数量，颜色按真实类别所在行归一化。

---

16. 指标手算与误分类分析

用 Binary 的商业类核对 F1

数量	值	对应含义
TP	129	商业新闻预测为商业。
FP	2	其他类别预测为商业。
FN	8	商业新闻预测成其他类别。

商业类 precision = 129/(129+2) = 98.47%；recall = 129/(129+8) = 94.16%；F1 = 2×129/(2×129+2+8) = 96.27%。将商业、政治和体育三个 F1 求平均，得到 Binary 的 Macro-F1 97.79%。总正确数 1,140 除以 1,152，得到 Accuracy 98.96%。

具体错误记录

原始行号	数据标签	BERT 预测
1832	business	politics
3161	sports	politics
10595	politics	business

原始行号	数据标签	BERT 预测
8198	sports	politics

样本 8198 的数据标签是 sports，BERT 预测为 politics。文章讨论一场比赛失利之后发生的犯罪事件，开头既有体育背景，也有犯罪事件线索，主题存在交叉。BERT 只读取开头部分，可能更容易受到其中犯罪事件线索的影响。

高权重词项

类别	Binary 正向权重最高的七个词
business	company, market, mr, companies, its, company's, flight
politics	mr, washington, republican, state, law, obama, officials
sports	players, league, game, football, season, team, player

football、league、team 与体育直接相关；washington、republican、obama 是政治类线索；company、market 与商业有关。mr 和 its 这样的通用词也在前列，说明分类器还利用了语料中的写作习惯。比较同一词在不同类别中的系数，可以进一步观察它对分类的作用。

---

17. 重复文本与测试分数的波动

主实验按原始数据随机划分，保留重复记录。为检查重复文本的影响，额外去掉测试集中已在训练集出现的新闻，保留 1,137 篇，使用原模型重新计算指标。下表并列列出原测试集和过滤后测试集的分数。

方法	原测试 Macro-F1	过滤后 Macro-F1
Binary BoW	97.79%	97.75%
Frequency BoW	97.52%	97.48%
GloVe 100d	96.68%	96.86%
Word2Vec AG	95.97%	95.89%
Word2Vec NYT	96.85%	97.04%
BERT	96.50%	96.44%

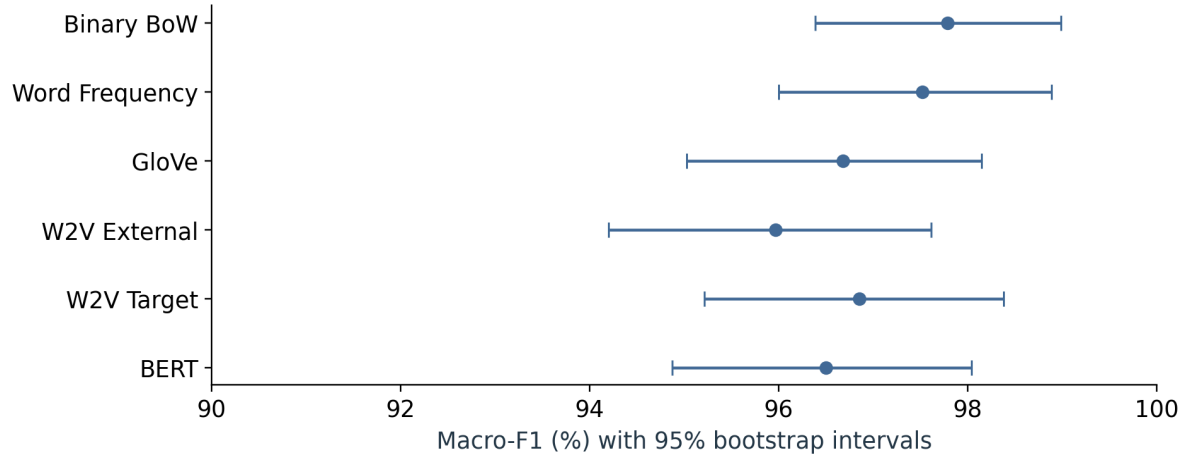


图 11 2,000 次分层 bootstrap 得到的 Macro-F1 95% 区间

bootstrap 在三个类别内部有放回抽样，保持各类别的原测试样本数，随机种子为 123。各模型使用同一次抽样行号，按配对方式计算差值。模型保持固定，区间反映测试样本变化带来的分数波动。

BERT 减去 Binary 的 Macro-F1 差值区间为 [-2.80, +0.16] 个百分点。由于置信区间包含 0，目前不能认为 Binary BoW 一定优于 BERT。此外，BERT 的输入限制为 64 个 token，也可能影响了结果。

---

18. 实验总结

这次实验中，最值得进一步分析的结果是 BERT 没有超过 Binary BoW。核对六组模型的测试行号和保存预测后，重新计算的指标与结果表一致。再查看输入长度，发现 99.91% 的测试新闻超过 64 个 token，BERT 实际上只读取了新闻开头，而词袋方法使用全文。这可能是本次两种方法表现不同的原因之一。

训练曲线也给出了一个具体观察：第三轮训练损失继续下降，验证 Macro-F1 却低于第二轮，因此最终采用第二轮模型。三组平均词向量中，NYT Word2Vec 表现最好，它的测试词项覆盖率也最高。后续可以尝试增加 BERT 输入长度，并用不同随机种子重复实验，观察这些结果是否稳定。

附录：以下是代码文件作用罗列

代码文件	作用
src/common.py	读取与划分数据、统一分词、评价指标、保存预测。
src/train_classical.py	两种词袋、三种词向量、Logistic Regression 选参。
src/train_bert.py	编码数据、三轮微调、验证选模型、最终测试。
src/predict.py	读取保存权重并预测单篇文本。
src/report_evidence.py	核算六组测试预测，并重新执行正确性检查。
src/report_charts.py	根据 JSON 结果生成九张数值图。
src/report_log_images.py	将实际日志摘录排版为配图。
src/build_report.py / report_details.py	整理报告正文，用 ReportLab 排版 PDF，并导出 Markdown。
tests/test_correctness.py	七项语义与结果检查。
run_experiments.py	完整实验入口。